```
$stmt = $pdo->query('SELECT * FROM test');
var_dump($stmt);
```

```
Output:
object(PDOStatement)#2 (1) {
  ["queryString"]=>
  string(18) "SELECT * FROM test"
}
```

The `query()` method returns an instance of `PDOStatement`, which we can use to iterate over the results. `SELECT * FROM test` simply means "return all fields for all rows in the `test` database." Here they are

```
foreach ($stmt as $item) {
    print "id: {$item['id']}, message: {$item['message']}" . PHP_EOL;
}
```

```
Output:
id: 1, message: some text
id: 2, message: some more text
id: 3, message: even more text
```

The `foreach ()` language construct accesses the `$stmt` object, which supports array access and returns one row for every loop iteration. These concepts are explained in detail in Chapter 5, "Array," and Chapter 4, "Logic and Control."

Prepared Statements

For additional security, it is advisable to never put user input directly into database queries. A malicious user could write their own SQL code, knowing it will be inserted into one of your own queries: these are known as *injection attacks*, and we'll look at them in more detail in Chapter 11, "Programming for the Internet." The `PDO` API has the facility to automatically sanitize user input in the form of *prepared statements*, and it's generally a good idea to just use them everywhere:

```
$stmt = $pdo->prepare('SELECT message FROM test WHERE id = :id');
$id = 3;
$stmt->bindParam(':id', $id);
```